

Unit-3

Robotics

Topic-1: Scaling

Scaling in robotics refers to the process of adapting and optimizing robotic systems to handle increased workloads, expand functionality, or integrate into larger systems without compromising efficiency or performance. Scaling involves addressing challenges in hardware, software, and deployment to ensure that robots meet evolving demands, whether in industrial automation, personal assistance, or complex autonomous systems.

Here's an overview of scaling in robotics across different dimensions:

1. Hardware Scaling

- **Increasing physical capacity:** Enhancing payload, speed, or reach of robots to accommodate larger tasks or environments.
 - Example: Moving from small robotic arms used in electronics assembly to larger ones capable of handling automotive parts.
- **Miniaturization:** Developing smaller robots for applications like medical surgeries or micromanufacturing.
- **Modular design:** Using interchangeable parts to easily upgrade or adjust robotic systems for specific tasks.
 - Example: Modular robot swarms that can reconfigure themselves for different tasks.

2. Software Scaling

- **Data processing:** Ensuring software can handle large volumes of sensory input and computational tasks.
 - Example: Scaling up AI algorithms for complex navigation in drones or autonomous vehicles.
- **Interoperability:** Designing software that integrates with other systems or scales across multiple robots in a fleet.
 - Example: Cloud-based control systems for coordinating warehouse robots.
- **Adaptive algorithms:** Machine learning and AI help robots improve their performance as tasks or environments change.

3. Fleet and Swarm Scaling

- **Coordination:** Managing fleets of robots to work collaboratively without conflicts.
 - Example: Hundreds of warehouse robots efficiently managing inventory in facilities like Amazon's fulfillment centers.
- **Communication:** Ensuring robust and scalable communication networks for real-time updates.
 - Example: Using 5G or edge computing to maintain low latency in multi-robot systems.
- **Decentralization:** Implementing systems where robots make independent decisions, reducing the load on central systems.

4. Manufacturing and Deployment Scaling

- **Cost-efficiency:** Reducing costs for mass production of robots to make them accessible for wider industries.
- **Automation:** Automating the manufacturing of robots themselves to meet increased demand.
- **Global deployment:** Developing systems that can adapt to diverse geographic, climatic, and regulatory conditions.

5. Challenges in Scaling Robotics

- **Hardware limitations:** As robots scale, maintaining durability, efficiency, and precision becomes challenging.
- **Energy demands:** Larger fleets or more capable robots need efficient power solutions (e.g., better batteries, renewable energy integration).
- **Complexity:** Managing increased complexity in software, coordination, and troubleshooting.
- **Safety and ethics:** Ensuring safe interactions between humans and scaled robotic systems.

Applications of Scaled Robotics

- **Logistics and Warehousing:** Large-scale deployment of robotic fleets for tasks like picking, packing, and transporting goods.

- **Healthcare:** Scaling robotic solutions for surgery, patient care, and drug delivery.
- **Construction:** Robots capable of building larger structures collaboratively.
- **Agriculture:** Robot swarms for planting, monitoring crops, and harvesting.
- **Exploration:** Robots scaled for space missions or underwater exploration.

Scaling in robotics is critical as industries increasingly rely on automation to meet efficiency, safety, and sustainability goals. Advances in technology, especially AI and materials science, play a pivotal role in enabling scalable robotic solutions.

Topic-2: Rotation

Rotation in robotics refers to the controlled movement of a robotic component or the entire robot around a specific axis. Rotations are critical in robotics for positioning end effectors (e.g., grippers, sensors) or orienting the robot in a workspace. Here's an overview:

Key Concepts of Rotation in Robotics

1. Axes of Rotation:

- Robots typically operate in a 3D space with rotations about the three primary axes:
 - **X-axis:** Pitch (up and down movement).
 - **Y-axis:** Roll (sideways rotation).
 - **Z-axis:** Yaw (spinning left or right).

2. Representations of Rotation: Different mathematical models are used to represent and compute rotations:

- **Euler Angles:**
 - Represent rotation using three angles (roll, pitch, yaw).
 - Simple but suffers from *gimbal lock* (a loss of one degree of freedom).
- **Rotation Matrices:**
 - 3×3 matrices representing rotation in 3D space.
 - Useful for transformations but computationally intensive.

- **Quaternions:**
 - Compact and efficient (4 numbers).
 - Avoids gimbal lock, commonly used in robotics.
- **Axis-Angle Representation:**
 - Rotation described by an axis and an angle of rotation about it.
 - Intuitive for understanding a single rotation.

3. **Forward and Inverse Kinematics:**

- Rotations are critical in kinematics:
 - **Forward Kinematics:** Calculates the position and orientation of the end effector given joint rotations and translations.
 - **Inverse Kinematics:** Determines the joint configurations required to achieve a desired position and orientation.

4. **Control and Actuation:**

- Rotational motion is achieved through actuators like servos, stepper motors, or continuous rotation motors.
- Feedback from sensors (e.g., encoders or gyroscopes) ensures precise control.

5. **Homogeneous Transformations:**

- Combines rotation and translation into a single 4×4 matrix.
- Used for efficient computation of transformations in robotic systems.

Applications of Rotation in Robotics

1. **Manipulator Arms:**

- Joints rotate to achieve desired end-effector positions.
- Rotations dictate degrees of freedom (DOF) in robotic arms.

2. **Mobile Robots:**

- Rotate about their center to navigate environments (e.g., differential drive systems).

3. **Aerial and Underwater Robotics:**

- Require precise control of pitch, roll, and yaw for stability and maneuverability.

4. Vision and Sensing:

- Cameras or sensors are rotated to capture different views or orientations.

5. Industrial Robotics:

- Perform tasks like welding or painting, requiring precise rotational movements.

Challenges in Robotic Rotation

1. Singularities:

- Points where the system loses certain degrees of freedom.
- Common with Euler angles.

2. Gimbal Lock:

- Loss of one degree of freedom in certain rotation sequences.

3. Accuracy:

- Imperfections in actuators or feedback systems affect rotational precision.

Topic-3: Translation

Translation refers to linear motion in space, where a robot or its components move in a straight line along a specific axis without rotation. This concept is vital in describing and programming robotic movements, particularly in areas like kinematics, path planning, and control systems.

Key Aspects of Translation in Robotics:

1. Axes of Translation:

- Robots often operate in a 3D space, so translation typically occurs along the **X**, **Y**, and **Z** axes.
- For example, a robotic arm may translate forward or backward along the X-axis, sideways along the Y-axis, or up and down along the Z-axis.

2. Degrees of Freedom (DOF):

- Translation is one type of degree of freedom (DOF). A robot may have separate DOFs for translation along each axis, depending on its design.

3. Applications:

- **Cartesian Robots:** These robots are specifically designed for translation along three perpendicular axes.
- **Pick-and-Place Tasks:** Robots translate to pick up an object from one position and place it in another.
- **Conveyor Systems:** Robots translate along a line to follow or interact with moving items.

4. Kinematics:

- Forward kinematics calculates the position of the robot's end-effector based on joint translations and rotations.
- Inverse kinematics determines the required translations (and rotations) of joints to achieve a desired end-effector position.

5. Linear Actuators:

- Devices like lead screws, pneumatic cylinders, or linear motors provide precise translation for robotic systems.

6. Sensors for Translation:

- Encoders, laser rangefinders, and inertial measurement units (IMUs) help measure and control translation.

7. Real-World Examples:

- A robotic vacuum cleaner translating across a room to clean its surface.
- An industrial robotic arm moving linearly to weld or assemble components.

Topic-4: Multiple Transformation

multiple transformations refer to applying a sequence of coordinate transformations to describe the motion or configuration of a robotic system. These transformations are crucial for understanding the position and orientation of

different parts of a robot (e.g., links, end-effectors) in relation to one another or to a global frame of reference.

Types of Transformations in Robotics

1. **Translation:** Shifting a point or object along the xxx, yyy, or zzz axes.
2. **Rotation:** Rotating an object about a specific axis (xxx, yyy, or zzz).
3. **Scaling** (rarely used): Changing the size of objects (not typically used for rigid robots).

Representation of Transformations

Transformations are typically represented using matrices:

- **Translation Matrix:** Adds offsets in Cartesian coordinates.
- **Rotation Matrix:** Represents rotations around axes.
- **Homogeneous Transformation Matrix:** Combines rotation and translation into a single 4×4 matrix for 3D transformations.

Multiple Transformations: Composition

In robotics, when a robot arm or mobile robot moves, multiple transformations are applied in sequence. For example:

- **Base to Joint 1**
- **Joint 1 to Joint 2**
- **Joint 2 to the End Effector**

Key Applications

1. **Forward Kinematics:** Calculates the position and orientation of the robot's end-effector using a chain of transformations from the base to the end-effector.
2. **Inverse Kinematics:** Determines the joint parameters required to achieve a desired end-effector position.
3. **Trajectory Planning:** Involves computing multiple transformations to guide the robot along a specific path.

Tools for Handling Transformations

- **Denavit-Hartenberg (DH) Parameters:** Simplifies the process of defining transformation matrices for serial robots.

- **Software Libraries:**
 - **Python:** Libraries like NumPy, SymPy, and ROS (Robot Operating System).
 - **MATLAB:** Robotics Toolbox.
 - **CAD Software:** For visualization and simulation.

Topic-5: Three Degree of Freedom

Degrees of freedom (DOF) refer to the number of independent motions a robotic system can perform. For a robotic system with **three degrees of freedom (3-DOF)**, it means it can move in three independent ways. These degrees of freedom could pertain to translations, rotations, or a combination of both, depending on the type of robot. Here's a breakdown:

1. Translation

- The robot can move linearly along one or more axes (e.g., x, y, z axes in a Cartesian coordinate system).
- For 3-DOF in translation, the robot can move along the **x**, **y**, and **z** directions.

2. Rotation

- The robot can rotate around one or more axes.
- For 3-DOF in rotation, the robot can rotate about the **x-axis (roll)**, **y-axis (pitch)**, and **z-axis (yaw)**.

3. Combination of Translation and Rotation

- A 3-DOF robot could perform a combination of linear and rotational movements. For example:
 - Translating along two axes (e.g., x and y) and rotating around one axis (e.g., z).
 - Translating along one axis (e.g., z) and rotating around two axes (e.g., x and y).

Examples of 3-DOF Systems:

1. 3-DOF Cartesian Robot:

- Moves linearly along the x, y, and z axes.
- Used in applications like 3D printing and CNC machines.

2. 3-DOF Articulated Robot:

- Typically consists of three rotational joints.
- Used for tasks like pick-and-place or painting.

3. 3-DOF Spherical Robot:

- Combines rotations and one linear movement to position its end-effector within a spherical workspace.

4. Drone Motion (Simplified to 3-DOF):

- A drone with motion constrained to yaw (rotation about the z-axis) and translations along x and y.

Topic-6: Forward and Reverse kinematics

Forward and Reverse Kinematics are fundamental concepts in robotics and animation, used to calculate the position and orientation of a robotic arm, mechanical system, or character joints.

Forward Kinematics (FK)

Forward kinematics is the process of calculating the **position and orientation of the end-effector** (e.g., a robotic arm's hand or tool) based on given joint parameters (angles or displacements).

Steps:

1. Start with known joint parameters (angles for revolute joints or distances for prismatic joints).
2. Use the geometry of the system and mathematical models (e.g., transformation matrices) to calculate the position and orientation of the end-effector.

Key Characteristics:

- Simple to compute when joint parameters are known.
- Deterministic and straightforward.

- Requires the **kinematic chain model**, which uses:
 - Link lengths
 - Joint constraints

Forward and Reverse Kinematics are fundamental concepts in robotics and animation, used to calculate the position and orientation of a robotic arm, mechanical system, or character joints.

Forward Kinematics (FK)

Forward kinematics is the process of calculating the **position and orientation of the end-effector** (e.g., a robotic arm's hand or tool) based on given joint parameters (angles or displacements).

Steps:

1. Start with known joint parameters (angles for revolute joints or distances for prismatic joints).
2. Use the geometry of the system and mathematical models (e.g., transformation matrices) to calculate the position and orientation of the end-effector.

Key Characteristics:

- Simple to compute when joint parameters are known.
- Deterministic and straightforward.
- Requires the **kinematic chain model**, which uses:
 - Link lengths
 - Joint constraints
 - Denavit-Hartenberg (D-H) parameters for transformation matrices.

Reverse (or Inverse) Kinematics (IK)

Inverse kinematics calculates the **joint parameters** needed to achieve a desired position and orientation of the end-effector.

Steps:

1. Start with the desired position and orientation of the end-effector.
2. Solve for the joint parameters (angles or displacements) using the system's kinematic model.

Key Characteristics:

- Complex and often requires numerical or iterative methods.
- May have **multiple solutions**, a **single solution**, or **no solution** (due to constraints or limitations).
- Solutions depend on:
 - Joint limits
 - Workspace constraints
 - Redundancy (if the robot has more degrees of freedom than needed)

Comparison

Aspect	Forward Kinematics	Reverse Kinematics
Input	Joint parameters	End-effector position/orientation
Output	End-effector position/orientation	Joint parameters
Complexity	Low	High
Use Case	Animation, path visualization	Control, goal-oriented motion

Applications

1. Forward Kinematics:

- Animation: Determining the position of character limbs.
- Robot simulation: Visualizing robotic motion.

2. Reverse Kinematics:

- Robotic arms: Picking and placing objects.
- Character rigging in games: Ensuring the character's hand reaches a specific point.

Topic-7: Robot Arm Dynamics

Robot arm dynamics is a branch of robotics and control systems that deals with the modeling, analysis, and control of robotic arms. It involves understanding the physical behavior of robotic manipulators in response to forces and torques, taking into account factors like mass, inertia, joint configurations, and the environment.

Here's an overview of key concepts in robot arm dynamics:

1. Kinematics

- **Forward Kinematics (FK):** Determines the position and orientation of the robot's end effector based on its joint parameters.
- **Inverse Kinematics (IK):** Computes the required joint parameters to achieve a desired position and orientation of the end effector.
- Kinematics does not consider forces or torques.

2. Dynamics

Robot dynamics incorporates forces, torques, and the physical properties of the arm.

Key Equations:

1. Newton-Euler Equations:

- Recursive formulation for computing forces and torques at each joint, considering linear and angular motion.

2. Lagrange Formulation:

- Utilizes energy principles (kinetic and potential energy) to derive the equations of motion.

3. Components in Dynamics

1. Mass/Inertia:

- Distributed mass of links affects dynamic behavior.
- The **inertia tensor** represents the resistance to angular acceleration.

2. Gravitational Forces:

- Affects torque requirements, especially for vertical motions.

3. Coriolis and Centrifugal Forces:

- Non-linear effects caused by joint velocities.

4. Friction:

- Static and dynamic friction at joints influence performance.

5. External Forces:

- Interaction forces from the environment (e.g., during contact tasks).

4. Control

To make the robotic arm achieve desired movements, dynamics models are used to design controllers:

- **Feedforward Control:**
 - Pre-computes torques required based on the dynamics model.
- **Feedback Control:**
 - Adjusts torques in real-time based on sensor data (e.g., PID, model predictive control).
- **Impedance Control:**
 - Balances position and force control for tasks involving physical interaction.
- **Adaptive Control:**
 - Adjusts control strategy based on changing dynamics or uncertainties.

5. Simulation and Implementation

- **Simulation Tools:**
 - MATLAB/Simulink, ROS/Gazebo, and PyBullet are commonly used to simulate robot arm dynamics.
- **Modeling Approaches:**
 - Denavit-Hartenberg (D-H) convention for kinematics.
 - Dynamic models based on software like CAD for accurate mass and inertia properties.

Example Use Cases:

1. Industrial robotic arms (e.g., pick-and-place tasks)
2. Surgical robots (e.g., precise motion control)
3. Autonomous manipulation (e.g., robots handling unknown objects)

Topic-8: D-H Representation of Robotics

The **Denavit-Hartenberg (DH) representation** is a standardized method used in robotics for modeling and describing the kinematics of robotic arms and other mechanisms. It provides a systematic way of representing the relationship

between the links of a robot, allowing for easier computation of forward and inverse kinematics.

DH Parameters

The following four transformation parameters are known as D–H parameters:

- **d** : offset along previous z to the common normal
- **θ** : angle about previous z from old x to new x
- **r** : length of the common normal (aka a , but if using this notation, do not confuse with α). Assuming a revolute joint, this is the radius about previous z .
- **α** : angle about common normal, from old z axis to new z axis

There is some choice in frame layout as to whether the previous x axis or the next x points along the common normal. The latter system allows branching chains more efficiently, as multiple frames can all point away from their common ancestor, but in the alternative layout the ancestor can only point toward one successor.

Topic-9: Basics of Trajectory Planning

Trajectory planning is a fundamental concept in robotics, autonomous vehicles, animation, and other fields that involve the movement of objects. It refers to the process of determining the path that an object or robot will follow over time, considering factors like position, velocity, acceleration, and constraints. Here's an overview of the basics:

1. What is a Trajectory?

- A **trajectory** is the path that an object follows in space over time, which is typically described as a series of positions (or states) at each point in time.
- In robotics, a trajectory represents the movement of a robot's end-effector, joints, or any mobile robot over time.

2. Components of a Trajectory

A trajectory usually includes:

- **Position (x, y, z)**: The coordinates in space at a given time.
- **Velocity (v)**: The rate of change of position with respect to time.
- **Acceleration (a)**: The rate of change of velocity with respect to time.

- **Time (t):** The time at each state of motion.

3. Types of Trajectory Planning

There are different types of trajectory planning, depending on the application:

- **Point-to-point (P2P):** The goal is to move from one point to another, typically used in robotic arms or point-to-point movements.
- **Continuous Path (CP):** Involves movement along a continuous curve, such as for CNC machines or robotic arms performing complex tasks.
- **Time-optimal Trajectory:** The goal is to move from one state to another in the shortest possible time, often used for fast motions.
- **Energy-optimal Trajectory:** Seeks to minimize the energy or effort required for movement, often used in systems with power limitations.

4. Mathematical Representation

- Trajectories can be described using **parametric equations**. Common methods of describing trajectories include:
 - **Polynomial functions:** e.g., cubic or quintic splines, used to interpolate between waypoints and ensure smooth motion.
 - **Piecewise Linear:** Connecting waypoints with straight lines, often used for simpler planning.
 - **Bézier curves:** Smooth curves that are useful for generating natural-looking paths in animation and robotics.
- These functions are chosen based on the type of motion required, constraints, and performance criteria (like smoothness or time efficiency).

5. Types of Constraints

Trajectory planning must consider various constraints to ensure the movement is physically feasible and safe:

- **Kinematic constraints:** Limits on position, velocity, and acceleration.
- **Dynamic constraints:** Forces and torques that the system can handle.
- **Environmental constraints:** Obstacles or boundaries in the environment.
- **Time constraints:** Desired time to reach a destination.

6. Optimization Methods

- **Trajectory optimization** aims to find the best trajectory under the given constraints.
 - **Linear programming** or **non-linear programming** can be used to solve optimization problems.
 - **Cost functions** are defined to minimize or maximize certain aspects (e.g., time, energy, smoothness).
- **Path planning** and **motion planning** are often combined to find feasible trajectories, especially in environments with obstacles.

7. Techniques Used in Trajectory Planning

- **Spline Interpolation**: Commonly used for smooth interpolation between waypoints.
- **Sampling-based methods** (e.g., Rapidly-exploring Random Tree (RRT)): Useful for high-dimensional planning spaces or environments with obstacles.
- **Dynamic Programming**: Used for planning when considering future states and their consequences.
- *A and Dijkstra's Algorithm**: Popular for grid-based pathfinding, though less common for continuous motion planning.

8. Applications of Trajectory Planning

- **Robotics**: Trajectory planning is used in robotic arms, mobile robots, and autonomous vehicles to ensure smooth, efficient, and safe movements.
- **Autonomous Vehicles**: In self-driving cars, trajectory planning ensures safe navigation through the environment, accounting for traffic, obstacles, and road conditions.
- **Animation and Simulation**: Used to animate characters or simulate physical systems, ensuring movements look realistic and natural.

9. Challenges in Trajectory Planning

- **Dynamic environments**: Dealing with obstacles or moving objects in real-time.
- **Complex constraints**: Managing the interplay of multiple constraints (time, energy, physical limits).

- **Computational complexity:** Real-time planning for systems with many variables and constraints can be computationally expensive.